

SFA2 — État de la détection pixel & isolation KOF98

Date : 2026-07-20

Branche : main

Contexte : Détection pixel pour SFA2 (SNES/snes9x — RAM inaccessible), sans casser KOF98 (RAM Neo Geo/FBNeo)

1. Architecture des deux chemins de détection

```
GameRunner.start()
├─ healthMemMap? (RAM config présente ?)
│   └─ OUI → startMemoryHealthReader()
│       └─ UDP_READ_CORE_RAM → processHealthFrame()
│           └─ p1Lost != null? → processLossCounters() → RETURN ← KOF98
│               └─ sinon → fallback santé heuristique (legacy)
└─ NON → startHealthBarCapture()
    └─ ffmpeg x11grab → analyzeHealthFrame() ← SFA2, SF2, ...
        state machine: WARMUP→PLAYING→KO_PENDING→KO_CONFIRMED→MATCH_END
```

Les deux chemins sont complètement séparés au niveau de la détection.

Ils partagent uniquement des variables de sortie : `p1Losses` , `p2Losses` , `matchEnded` , `roundNumber` , `matchPerfectKos` .

2. KOF98 — Chemin RAM (FBNeo) : PROTÉGÉ

Ce qui est en place

Détection	Mécanisme	Adresse(s) RAM	Statut
Santé P1/P2	UDP `READ_CORE_RAM`	`0x8238` / `0x8438`	✓ Stable
Timer 16-bit	UDP (timer-only fast poll)	`0xA83A-A83B`	✓
Perso actif P1/P2	Lecture RAM	`0x8256` / `0x8456`	✓
Mode gauge	Lecture RAM	`0x821E` / `0x841E`	⚠ Adresses pas encore validées
Équipes (3 persos)	Lecture RAM + freeze	`0xA84E`/`0xA84F`/`0xA851` (P1)	✓ Figées au combat
Ordre de sélection	Lecture RAM one-shot	`0x15CB`/`0x15CA`/`0x15CD` (P1)	✓ Capturé, pas encore câblé dans overlay
Compteur de pertes	**Autoritaire**	***`0xA859` (P1) / `0xA868` (P2)**	✓ **C'est ça qui drive tout**
Round gagné/perdu	`processLossCounters()`	Diff `prevLost` → `currLost`	✓
Time-over	`memTimer16 > 0` → `0` transition	`0xA83A`	✓
KO	Perte incrémentée sans time-over	Compteurs	✓

Détection	Mécanisme	Adresse(s) RAM	Statut		
Perfect KO	<code>roundMinHealth ≥ 95%</code> (combat stable)	Heuristique (pas de compteur RAM fiable)	✓		
Draw	Les deux compteurs incrémentés au même poll	<code>0xA859</code> + <code>0xA868</code>	✓		
Fin de match	<code>p1Lost ≥ 3</code>		<code>p2Lost ≥ 3</code>	Hardcodé à 3	✓
Continue/Revanche	Pièce via <code>0xF2C0</code> , START loop	UDP <code>btn</code> commands	✓		

Pourquoi KOF98 n'est pas affecté par les changements SFA2

- `processHealthFrame()` (ligne 1422) : Si `healthMemMap?.p1Lost != null`, appelle `processLossCounters()` et **return immédiatement** — le code pixel n'est jamais atteint
- `analyzeHealthFrame()` est appelé uniquement depuis le handler `ffmpeg.stdout.on("data")` — qui n'est jamais démarré pour KOF98 (car `startMemoryHealthReader()` est appelé à la place)
- `pixelConfig` vaut `null` pour KOF98 (`getPixelConfig("kof98.zip")` → pas d'entrée dans `PIXEL_GAME_CONFIGS`)
- Match end KOF98** : `processLossCounters()` ligne 1397 → `p1Lost >= 3 || p2Lost >= 3` (hardcodé, unchanged)

⚠ Points d'attention KOF98

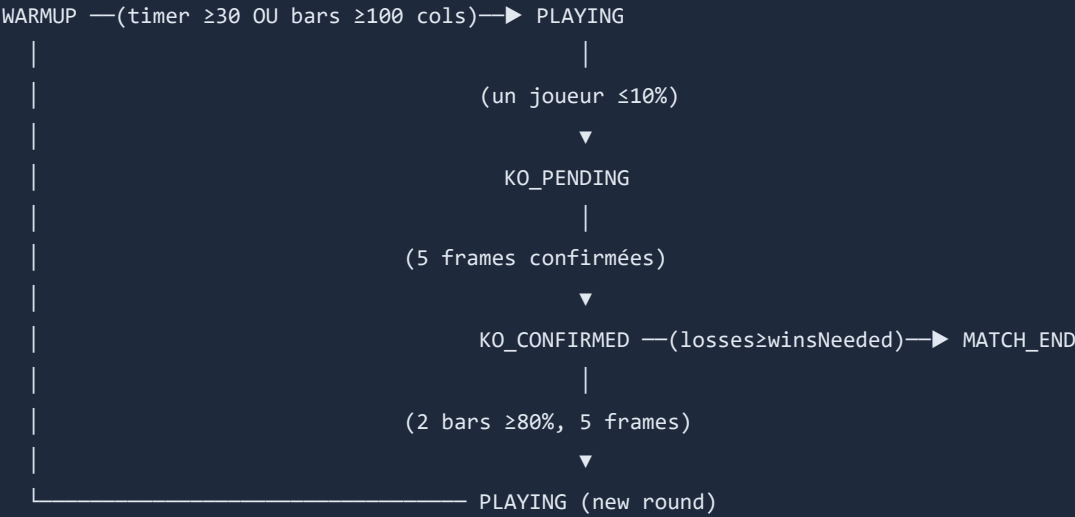
- Mode gauge ADVANCED/EXTRA** : les adresses `0x81F0` / `0x83F0` étaient fausses, remplacées par `0x821E` / `0x841E` dans la config DB — **pas encore validées live**
- Ordre de sélection** : adresses trouvées et validées (`0x15CB` / `0x15CA` / `0x15CD`), mais **pas encore câblé** dans `matchMeta()` pour l'overlay de fin de match
- `winsNeeded` **hardcodé à 3** dans `processLossCounters()` — si un jour on veut du KOF98 en BO1, il faudra paramétrer

3. SFA2 — Chemin pixel (snes9x) : EN COURS 🛠

Config par ROM

```
PIXEL_GAME_CONFIGS = {
  "Street Fighter Alpha 2 (Europe).sfc": {
    stripeY: 110, stripeH: 24,      // barre de vie = 24px à y=110 (après bordure noire)
    p1StartX: 70,  p1EndX: 310,    // barre P1 à gauche
    p2StartX: 450, p2EndX: 768,    // barre P2 à droite
    winsNeeded: 2,                  // best-of-3 (SF2 rules)
    timer: {
      digits: [bitmaps 0-9],       // templates 8x12 bits
      leftDigitX: 338, rightDigitX: 362,
      digitW: 22, digitH: 24,
      minBrightRatio: 0.15,
    },
  },
};
```

State machine (GamePhase)



Ce qui est fait (commits récents — 2026-07-20)

Détection	Mécanisme	Fichier	Statut
Santé P1/P2	Column scan + médiane glissante	`pixel-match-analyzer.ts`	✅ Stable
Barre calibrée	`p1FullBarWidth` max warmup + recalibration timer-start	`pixel-match-analyzer.ts`	✅

Détection	Mécanisme	Fichier	Statut
Seuil KO	<code>`KO_THRESHOLD = 10%`</code>	<code>`pixel-match-analyzer.ts`</code>	✓
Confirmation KO	5 frames (<code>`KO_CONFIRM_REQUIRED`</code>)	<code>`pixel-match-analyzer.ts`</code>	✓
Grâce début round	16 frames (<code>`PLAYING_GRACE_FRAMES`</code>)	<code>`pixel-match-analyzer.ts`</code>	✓
KO	KO_PENDING → KO_CONFIRMED + retroactive KO	<code>`pixel-match-analyzer.ts`</code>	✓
Perfect KO	Ratio-based : <code>`minFilled/maxFilled ≥ 0.95`</code> (colonnes brutes)	<code>`pixel-match-analyzer.ts`</code>	✓
Draw	Double KO simultané, santé égale	<code>`pixel-match-analyzer.ts`</code>	✓
Nouveau round	2 barres ≥ 80% pendant 5 frames	<code>`pixel-match-analyzer.ts`</code>	✓
Fin de match	<code>`losses ≥ winsNeeded`</code> (2 pour SFA2)	<code>`pixel-match-analyzer.ts`</code>	✓
Timer OCR	Template matching 8×12 → digits 0-99	<code>`pixel-match-analyzer.ts`</code>	✓
Time-over	timer→0 dans PLAYING armé → compare santé → roundResult	<code>`pixel-match-analyzer.ts`</code>	✓ Validé live
Bars-vanished KO	Double drop à 0% avec lastRunning sain → KO rétroactif	<code>`pixel-match-analyzer.ts`</code>	✓
match_state WebSocket	PixelMatchAnalyzer "health" → GameRunner "matchState" → ws-handler	<code>`game-runner.ts`</code> / <code>`ws-handler.ts`</code>	✓
Warmup rapide	<code>`fastWarmup = true`</code> → 8 frames	<code>`pixel-match-analyzer.ts`</code>	✓
Anti-fantôme KO	Gardes timer-decrease, thaw détection, KO recovery	<code>`pixel-match-analyzer.ts`</code>	✓
Portrait capture	ImageMagick <code>`import -depth 8 -window root`</code> → PPM 8-bit	<code>`game-runner.ts`</code>	✓

Détection	Mécanisme	Fichier	Statut
Portrait calibrator	Collecte 18 échantillons/match, consensus ≥10/perso	`game-runner.ts`	✅ (collecte OK, pas assez de matchs)

Ce qui MANQUE pour SFA2

Détection	Problème	Priorité
Templates portrait	Calibrateur collecte mais 1 seul échantillon/match → besoin de 10 matchs ou persistance disque. Script `generate-portrait-templates.mjs` prêt mais colonnes droites tout-noir (grille à ajuster).	●
Tests live SFA2	Tout le code pixel est testé en CPU auto-fight. Fonctionne 2/3 du temps (timer instable sur ~1/3 des runs).	●
Autres jeux SNES	La structure `PIXEL_GAME_CONFIGS` est prête. Ajouter un jeu = une entrée.	●
Config charger depuis DB	Actuellement hardcodé dans `pixel-game-config.ts`.	●

4. Reste du chantier (autres fichiers modifiés)

DB & Config — db.ts

- ✅ Tables `duel_games` , `duel_game_modes` , `duel_game_controls` , `duel_game_config_versions`
- ✅ Seed 4 jeux × 3 modes (standard/XL/fighter)
- ✅ Règles multilingues FR/EN
- ✅ RAM config KOF98 + KOF2002 stockée en JSON
- ⚠️ Config pixel SFA2 pas encore en DB (hardcodé dans `PIXEL_GAME_CONFIGS`)

Flow règles — `respond/route.ts` + `confirm-rules/route.ts`

- ✅ Accept → `rules_pending` (ne crée plus la session directement)
- ✅ Nouvelle route `confirm-rules` (fichier untracked)
- ✅ Notifications aux deux joueurs

- ☒ Colonnes `challenger_rules_accepted` / `target_rules_accepted`

UI Duel — `DuelLobby.tsx` , `DuelScoreHUD.tsx` , **etc.**

- ☒ Filtre joueurs, lien invitation, recovery après refresh
- ☒ Composants : `DuelScoreHUD` , `DuelRulesOverlay` , `DuelPauseOverlay` , `DuelGameSelector` , `DuelModeSelector` , `DuelWizardStepper`
- ☒ i18n étendu (sections règles, pause, wizard, mode)
- ☒ Frais d'entrée dynamiques par jeu/mode

game-server — `config.ts` , `docker-compose`

- ☒ Support SNES (core snes9x, boutons, résolution)
- ☒ Variables Turso dans Docker

5. Plan d'action recommandé

Étape 1 — Time-over SFA2 (le vrai trou)

```
Dans analyzeHealthFrame(), état PLAYING :
- Lire le timer via readTimerFromFrame()
- Si lastTimerValue passe de >0 à 0 :
  → comparer p1Health vs p2Health
  → émettre roundResult avec koType: "timeout"
  → incrémenter p1Losses ou p2Losses
  → passer en KO_CONFIRMED
```

Étape 2 — Valider live

- Lancer un combat SFA2, laisser le timer s'épuiser
- Vérifier que le time-over est bien détecté
- Vérifier KO, perfect KO, draw

Étape 3 — Nettoyer et committer

- Une fois la détection SFA2 complète, committer par lots logiques
- Ne pas mélanger les changements game-runner avec les changements UI

Étape 4 — KOF98 gauge mode

- Valider les adresses `0x821E` / `0x841E` par diff live (ADVANCED vs EXTRA)

6. Résumé visuel

```

GameRunner.start()

healthMemMap? (RAM config from DB or hardcoded)
├── OUI (KOF98, KOF2002)
│   ├── startMemoryHealthReader()
│   │   └── UDP READ_CORE_RAM every 250ms
│   │       ↓
│   │   processHealthFrame()
│   │       ├── p1Lost != null? → processLossCounters()
│   │       │   ├── ✓ Rounds   ✓ KO   ✓ Time-over
│   │       │   ├── ✓ Perfect  ✓ Draw  ✓ Match end
│   │       │   └── ⚠ Gauge mode (adresses à valider)
│   └── NON (SFA2, SF2, autres SNES)
│       ├── startHealthBarCapture()
│       │   └── ffmpeg x11grab 2fps → stripe santé
│       │       ↓
│       │   analyzeHealthFrame()
│       │       State machine:
│       │       ├── ✓ Santé (column scan + médiane)
│       │       ├── ✓ KO (4-frame confirm)
│       │       ├── ✓ Perfect KO (minHealth ≥ 95%)
│       │       ├── ✓ Draw (double KO simultané)
│       │       ├── ✓ Nouveau round (2 barres ≥ 80%)
│       │       ├── ✓ Match end (winsNeeded configurable)
│       │       ├── ✓ Timer OCR (digits lus)
│       │       └── ✗ Time-over (timer pas utilisé!)

```